Capstone Project
Mid-Semester Evaluation Report

on

Automated Real-Time Hydroponics Telemetry and Alerting System

PulsePonics: Software Platform
(Messaging, Backend, Database and Web Dashboard)

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology
in
Biotechnology

Submitted by

Nitish Kumar Sah
123BT0791

Under the Guidance of

Prof. A. Thirugnanam
Department of Biotechnology and Medical Engineering



Department of Biotechnology and Medical Engineering
National Institute of Technology Rourkela
Odisha – 769008

October 2026

Certificate

This is to certify that the mid-semester report titled **"Automated Real-Time Hydroponics Telemetry and Alerting System"**, submitted by **Nitish Kumar Sah** (Roll No. 123BT0791) to the Department of Biotechnology and Medical Engineering, National Institute of Technology Rourkela, is a record of bona fide work carried out by him under my supervision as part of the Capstone Project of the Bachelor of Technology programme in Biotechnology.

The work presented in this report covers the software platform of the project and is submitted for the mid-semester evaluation. To the best of my knowledge, it has not been submitted elsewhere for the award of any degree or diploma.

Place: Rourkela
Date: _____

_____
Prof. A. Thirugnanam
Department of Biotechnology and
Medical Engineering
NIT Rourkela

Declaration

I hereby declare that this report titled **"Automated Real-Time Hydroponics Telemetry and Alerting System"**, submitted for the mid-semester evaluation of the Capstone Project, is a record of authentic work carried out by me under the guidance of Prof. A. Thirugnanam, Department of Biotechnology and Medical Engineering, National Institute of Technology Rourkela.

I further declare that this report, or any part thereof, has not been submitted by me or any other person for the award of any other degree or diploma of this or any other institute, and that all sources of information used in its preparation have been duly acknowledged.

Place: Rourkela, Odisha
Date: October 30, 2026

Nitish Kumar Sah
123BT0791
NIT Rourkela

Acknowledgments

I would like to express my sincere gratitude to my project guide, **Prof. A. Thirugnanam**, Department of Biotechnology and Medical Engineering, NIT Rourkela, for his guidance, encouragement and valuable feedback throughout the first half of this capstone project. His suggestions helped shape both the scope of the system and the way it has been engineered and evaluated.

I am thankful to the **Department of Biotechnology and Medical Engineering, National Institute of Technology Rourkela**, for providing the opportunity and the academic framework to undertake an interdisciplinary project that brings together plant cultivation and modern software and embedded engineering.

Finally, I thank my family and friends for their constant support and encouragement.

Nitish Kumar Sah

Abstract

Hydroponic cultivation depends on keeping the nutrient solution inside narrow chemical and thermal ranges; pH, electrical conductivity (EC) and water temperature drift continuously and are traditionally checked by hand. This capstone project builds **PulsePonics**, an Internet-of-Things system that measures these parameters with an ESP32 sensor node, transmits them in real time, stores them, and alerts the grower when they leave safe limits. This mid-semester report documents the complete *software platform* of the system.

The platform consists of an Eclipse Mosquitto MQTT broker with authenticated device access, a Go backend that ingests telemetry from MQTT and HTTP, a MySQL time-series store, a per-device threshold and alerting engine, a WebSocket push channel, and a React/TypeScript dashboard. Users register their own devices, and every request is authenticated with short-lived JSON Web Tokens and rotating refresh tokens. The ingestion pipeline is idempotent under MQTT's at-least-once delivery, survives broker restarts, and rejects telemetry from unregistered devices. Alerts use hysteresis so that sensor noise at a limit does not cause flapping, and long history ranges are downsampled in the database before reaching the browser. The whole stack is containerised with Docker Compose, with automatic HTTPS for cloud deployment.

On a development machine, the backend served typical API requests with a median latency of 1–6 ms, and a reading published over MQTT reached the dashboard's WebSocket in 1.9 ms (median of 200 messages). A six-hour test series of 2,160 readings with an injected pH fault produced exactly one alert, opened and resolved at the expected points. 106 automated checks (unit and end-to-end) pass.

Keywords: Hydroponics, Internet of Things, ESP32, MQTT, Go, MySQL, WebSocket, React, JWT authentication, Docker, real-time alerting.

Contents

Certificate ii

Declaration iii

Acknowledgments iv

Abstract v

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives . . . 1
- 1.4 Scope of this Report . . . 1

2 Methodology 2
- 2.1 Overall Approach . . . 2
- 2.2 System Architecture . . . 2
- 2.3 Technology Stack . . . 2
- 2.4 Data Model . . . 2
- 2.5 Authentication and Authorization Design . . . 3
- 2.6 Telemetry Ingestion Pipeline . . . 4
- 2.7 Alerting Engine Design . . . 4
- 2.8 Time-Series Query Strategy . . . 5

3 System Implementation 5
- 3.1 Backend Structure and API Design . . . 5
- 3.2 Input Validation and Data Integrity . . . 5
- 3.3 Resilient MQTT Subscription . . . 6
- 3.4 Alert Decision Logic . . . 6
- 3.5 Time-Bucketed Aggregation . . . 7
- 3.6 Real-Time Delivery . . . 7
- 3.7 Frontend Dashboard . . . 8
- 3.8 Containerisation and Deployment . . . 8
- 3.9 Testing and Quality Assurance . . . 8

4 Result 9
- 4.1 Summary of Deliverables . . . 9
- 4.2 Telemetry Visualisation and Alert Behaviour . . . 9
- 4.3 API and End-to-End Performance . . . 9
- 4.4 Reliability and Data Integrity . . . 11
- 4.5 Dashboard . . . 11
- 4.6 Test Results . . . 11

5 Discussion **13**
5.1 Engineering Reliable Ingestion 13
5.2 Design Trade-offs Considered 13
5.3 Limitations 13

6 Conclusion **13**
6.1 Future Scope 14

References **14**

List of Figures

2.1 High-level architecture of the PulsePonics platform 2
2.2 Sequence of a telemetry reading from the ESP32 to the dashboard 4
2.3 Alert state machine with hysteresis (per device and metric) 4
4.1 Six hours of telemetry as served by the history API (green: threshold band; red: interval during which the pH alert was open) 10
4.2 Measured response times of key operations (300 requests each; 200 messages end to end) 10
4.3 Device dashboard with live readings, history chart and an open pH alert 11
4.4 Threshold configuration and responsive design of the dashboard 12

List of Tables

2.1 Technology stack used in the project 3
2.2 Database schema overview 3
3.1 Summary of implemented API endpoint categories 5
4.1 Summary of key results achieved 9
4.2 Automated test results 12

Chapter 1

Introduction

1.1 Background

Hydroponics grows plants without soil, supplying nutrients through a recirculating water solution. It achieves high yields with less water and land, but it leaves the crop with almost no buffer against mistakes in the solution chemistry. For common leafy greens the solution is typically held at a pH of 5.5–6.5, an EC of about 1.2–2.4 mS/cm and a water temperature of 18–24 °C [1]. Outside these ranges nutrient uptake drops, roots are stressed, and losses can occur within hours.

In small and academic setups these parameters are still measured manually with handheld meters a few times a day. Drift between checks goes unnoticed, readings are rarely recorded systematically, and there is no history to diagnose problems from.

1.2 Problem Statement

The project aims to replace manual monitoring with an automated pipeline that (i) measures the critical parameters continuously at the tank, (ii) delivers them reliably to a central server despite unreliable Wi-Fi, (iii) stores them as a queryable time series, (iv) presents them live to the grower from anywhere, and (v) raises an alert as soon as a parameter breaches a safety threshold.

1.3 Objectives

Following the project proposal, the objectives of the project are:

1. Build an ESP32 edge device that reads pH, EC, water temperature and air conditions, using a 16-bit ADS1115 ADC for the analog probes.
2. Transport telemetry over MQTT with acknowledged (QoS 1) delivery.
3. Develop a Go backend that validates, stores and serves time-stamped telemetry from MySQL through secure, paginated REST endpoints.
4. Develop a real-time React dashboard with historical charts over selectable time ranges.
5. Implement an alerting engine that detects threshold breaches of critical parameters and notifies the user.

1.4 Scope of this Report

The system has four layers: hardware, transport, backend with database, and presentation. This mid-semester report covers the **software platform**: the transport, backend, database and presentation layers, together with authentication, deployment and testing, all of which are complete. On the hardware side, the ESP32 firmware already reads water temperature (DS18B20), synchronises time over NTP and publishes over MQTT with credentials. The BME280 and ADS1115 drivers and pH/EC calibration are scheduled for the second half. The telemetry analysed in Chapter 4 was therefore generated by a simulator that speaks exactly the same MQTT/JSON contract as the firmware, and it passed through the full production pipeline.

Chapter 2

Methodology

2.1 Overall Approach

The work followed the sprint plan of the proposal, with one rule: the **firmware's documented MQTT contract is the source of truth**. The payload fields, units, accepted ranges and topic names defined in the hardware documentation were fixed first, and the backend was built to accept that payload unchanged. This lets the hardware and software progress independently and allows a simulator to stand in for the device. Logic that is easy to get subtly wrong (input validation, alert decisions, token rotation, time bucketing) was written as pure functions with unit tests. Every feature was then verified end to end against the running containerised stack.

2.2 System Architecture

Figure 2.1 shows the deployed architecture. The ESP32 publishes JSON readings to Mosquitto. The Go backend is the broker's only subscriber: it validates and stores each reading in MySQL, evaluates the device's thresholds, and pushes the reading and any alert change to the owner's open dashboards over a WebSocket. The broker, backend and database run as Docker containers on one AWS EC2 instance. Caddy terminates HTTPS with an automatically issued Let's Encrypt certificate, so the React dashboard, served from Vercel over HTTPS, can call the API and open a secure WebSocket (`wss://`).

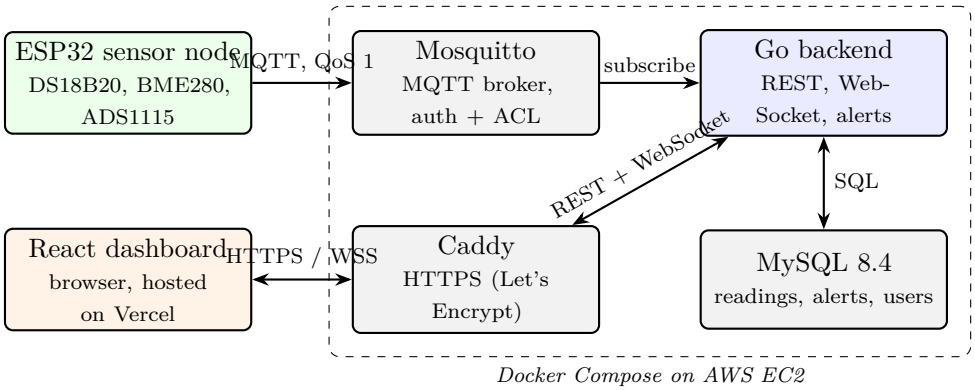


Figure 2.1: High-level architecture of the PulsePonics platform

2.3 Technology Stack

Table 2.1 lists the technologies used in each layer. Go was chosen for the backend for its low memory footprint on a small cloud instance and its first-class concurrency, which suits a service that runs an MQTT consumer, an HTTP server and many WebSocket writers at the same time.

2.4 Data Model

The schema (Table 2.2) has six tables. All timestamps are stored in UTC: the database session and the driver are both pinned to UTC, so the data does not shift when the server's time zone changes. Every child table references its device with `ON DELETE CASCADE`, so deleting a device or an account removes its data completely. Two constraints carry business rules directly in the database: `UNIQUE(device_id, recorded_at)` on `readings` makes ingestion idempotent, and on `alerts` a generated column `open_marker` (1 while the alert is open, `NULL` once resolved) is part of a unique key with device and metric. Because `NULL`s never collide in a unique index, this allows

Table 2.1: Technology stack used in the project

Layer	Technology
Edge device	ESP32 (C++, Arduino framework, PlatformIO); DS18B20, ADS1115, BME280
Transport	MQTT 3.1.1 over TCP; Eclipse Mosquitto 2 with password file and ACL
Backend	Go (standard `net/http` router), sqlx, Eclipse Paho MQTT, gorilla/websocket, golang-jwt, bcrypt
Database	MySQL 8.4 (InnoDB), embedded versioned SQL migrations
Frontend	React 19, TypeScript, Vite, Tailwind CSS 4, React Router, Recharts
Infrastructure	Docker Compose, Caddy 2 (automatic HTTPS), AWS EC2, Vercel
Testing	`go test` with race detector, Vitest, Unity (firmware), Bash end-to-end suite

at most one *open* alert per device and metric while keeping any number of resolved ones.

Table 2.2: Database schema overview

Table	Key columns and constraints	Purpose
`users`	unique email, bcrypt password hash	user accounts
`refresh_tokens`	SHA-256 token hash (unique), expiry, revoked, rotated flag	session rotation and reuse detection
`devices`	unique `device_id`, owner FK, API-key hash	device registry and ownership
`readings`	six nullable DECIMAL sensors, `recorded_at`; unique (device, time)	telemetry time series
`thresholds`	PK (device, metric), min, max, enabled	per-device alert limits
`alerts`	metric, kind, trigger/latest value, opened/resolved/acknowledged	alert history

2.5 Authentication and Authorization Design

Users sign up with an email and password, stored as a bcrypt hash [6]. Login returns a pair of tokens:

- a **15-minute access token**: an HS256-signed JWT [4], sent as a bearer token and checked without a database lookup;
- a **7-day refresh token**: an opaque random string, stored only as its SHA-256 hash.

Each refresh rotates the token. If a token that has already been rotated is presented again, it must have been copied, so every session of that user is revoked. A password change likewise revokes all other sessions.

Authorization is ownership-based. Every device belongs to exactly one user, and every device-scoped query checks it. Another user's device returns `404` rather than `403`, so device IDs cannot be probed.

Devices authenticate separately:

- to the broker with an MQTT username and password, restricted by an ACL to publishing on `hydroponics/+/telemetry` and `.../status`;
- for HTTP ingestion, with a per-device API key that is stored hashed and shown only once.

Login and signup are rate-limited per IP address, and browser access is restricted to an allowlist of origins, both for REST calls (CORS) and for WebSocket upgrades.

2.6 Telemetry Ingestion Pipeline

Figure 2.2 traces one reading through the system. MQTT QoS 1 guarantees *at-least-once* delivery: the firmware re-sends a reading when the broker's acknowledgment (PUBACK) is late, so the same reading can arrive twice. The backend makes this safe by relying on the unique `(device_id, recorded_at)` key. A duplicate is logged and dropped, and is neither broadcast nor evaluated for alerts. Readings from device IDs that no user has registered are discarded, and a message whose payload `device_id` differs from its topic is rejected. A device can therefore only write its own data.

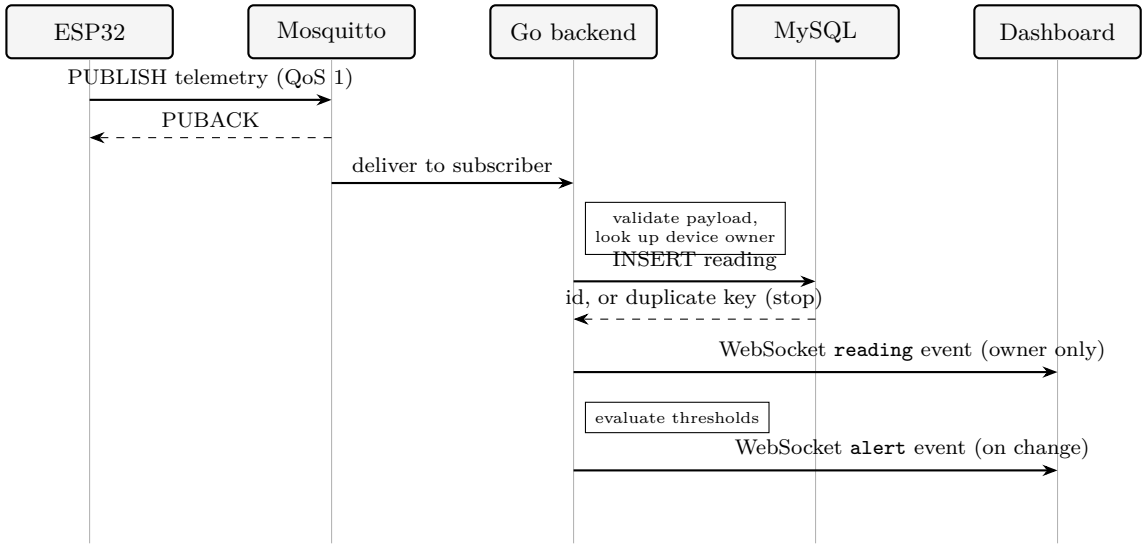


Figure 2.2: Sequence of a telemetry reading from the ESP32 to the dashboard

2.7 Alerting Engine Design

Each device has a minimum and maximum per metric. Defaults are pH 5.5–6.5, EC 1.2–2.4 mS/cm, water 18–24 °C, air 18–28 °Cand humidity 45–75 %; pressure is monitored without limits. The user can edit any of these. Every new reading is checked against these limits, and an alert moves through two states (Figure 2.3).

A naive rule ("resolve as soon as the value is back in range") makes a noisy sensor sitting on a limit open and close an alert on every reading. The engine therefore applies *hysteresis*: an alert resolves only once the value is back inside the band by a margin of $\alpha = 5\,\%$ of the band's width. For a high-side alert the resolve condition is

$$v \;\le\; v_{\max} - \alpha\,(v_{\max} - v_{\min}), \qquad \alpha = 0.05, \tag{2.1}$$

and symmetrically for a low-side alert. A `null` sensor value leaves the alert unchanged, because a missing reading is not evidence that the fault has cleared.

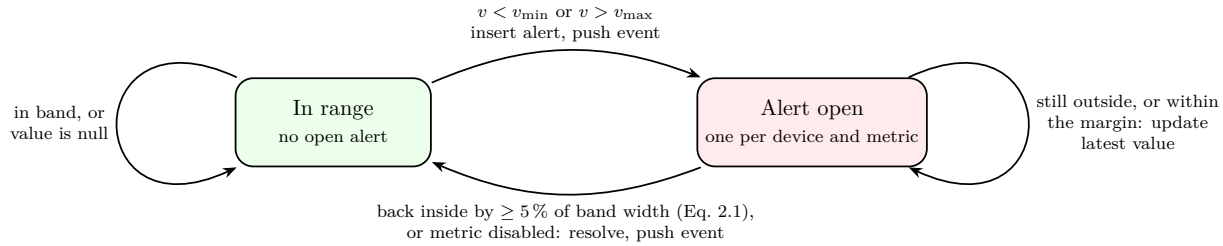


Figure 2.3: Alert state machine with hysteresis (per device and metric)

2.8 Time-Series Query Strategy

At the firmware's development interval of 10 s, a device produces 8,640 readings per day, far more than a chart needs and slow to send to a phone. The history endpoint therefore returns at most N points (default 500). If the requested range R contains more readings than that, MySQL averages them into epoch-aligned buckets of size

$$b = \min\left\{ s \in S \;:\; s \geq \left\lceil \frac{R}{N-1} \right\rceil \right\}, \quad S = \{10, 15, 30, 60, 120, 300, \ldots, 86400\} \text{ s.} \tag{2.2}$$

Dividing by $N - 1$ instead of N accounts for a range that straddles one extra epoch-aligned bucket, so the result never exceeds N points. Choosing b from a set of round values keeps chart timestamps on whole minutes.

Chapter 3

System Implementation

3.1 Backend Structure and API Design

The backend is organised in vertical feature slices (`auth`, `devices`, `readings`, `alerts`), each with its own model, repository, service and HTTP handler. Shared platform packages provide typed errors, JSON helpers, middleware, database access, migrations and the WebSocket hub. Services return typed errors that a single helper maps to HTTP status codes, so internal details never leak to clients. Table 3.1 summarises the 24 routes; all of them are under `/api`.

Table 3.1: Summary of implemented API endpoint categories

Category	Routes	Purpose
Authentication and account	7	signup, login, refresh, logout, profile, change password, delete account
Device registry	6	list, register, get, rename, delete, rotate API key
Readings	5	HTTP ingestion (device key), auto-aggregated history, latest, recent, metric metadata
Thresholds and alerts	4	read and update thresholds, list alerts, acknowledge
System	2	health check, authenticated WebSocket stream
Total	**24**	

3.2 Input Validation and Data Integrity

Both ingestion paths (MQTT and HTTP) call the same `Ingest` service, which starts with the validation in Listing 3.1. The accepted ranges mirror the firmware's own limits. The firmware already sends out-of-range values as `null`, so genuine readings are never rejected while corrupted ones are. A timestamp more than five minutes in the future, which indicates an unsynchronised device clock, is refused; otherwise that reading would remain the "latest" forever.

Listing 3.1: Payload validation against the hardware contract (`readings/validate.go`)

```
func validate(in *Input, now time.Time) error {
    if err := devices.ValidateDeviceID(in.DeviceID); err != nil {
        return err // 3-64 chars, no MQTT wildcards or '/'
    }
    any := false
    for _, m := range Metrics { // ranges mirror the firmware's include/config.h
        v := in.value(m.Key)
        if v == nil {
            continue // null = sensor missing, failed or uncalibrated
```

```
        }
        any = true
        if *v < m.Min || *v > m.Max {
            return apperr.Invalid(m.Key, fmt.Sprintf("%s must be between %s and %s",
                m.Key, fmtNum(m.Min), fmtNum(m.Max)))
        }
    }
    if !any {
        return apperr.Invalid("", "at least one sensor value must be provided")
    }
    if in.RecordedAt.IsZero() {
        in.RecordedAt = now
    }
    if in.RecordedAt.After(now.Add(maxClockSkew)) { // 5 minutes
        return apperr.Invalid("recorded_at", "recorded_at is in the future; check the device clock")
    }
    in.RecordedAt = in.RecordedAt.UTC().Truncate(time.Millisecond) // matches DATETIME(3)
    return nil
}
```

3.3 Resilient MQTT Subscription

With a clean MQTT session, the broker forgets a client's subscriptions when its connection drops. An early version of the backend subscribed only once at startup. It reconnected after a broker restart but silently stopped receiving telemetry while reporting itself healthy. The subscription is therefore issued in the on-connect handler, which runs after every reconnection (Listing 3.2). Connection retry is also enabled, so the backend waits for a broker that starts after it instead of exiting.

Listing 3.2: MQTT client set-up and message outcome handling (`readings/mqtt_subscriber.go`, condensed)

```
opts := paho.NewClientOptions().
    AddBroker(cfg.BrokerURL).
    SetUsername(cfg.Username).SetPassword(cfg.Password).
    SetAutoReconnect(true).
    SetConnectRetry(true). // keep retrying if the broker is not up yet at boot
    SetConnectRetryInterval(5 * time.Second)

// Subscribe on EVERY (re)connect: a clean session loses subscriptions on disconnect.
opts.SetOnConnectHandler(func(c paho.Client) {
    c.Subscribe("hydroponics/+/telemetry", 1, s.handleMessage)
})

// inside handleMessage, after checking payload device_id == topic device_id:
reading, err := s.service.Ingest(ctx, in)
switch {
case err == nil:
    log.Printf("mqtt: stored reading id=%d device=%s", reading.ID, reading.DeviceID)
case errors.Is(err, ErrUnknownDevice):
    log.Printf("mqtt: device %q is not registered, reading dropped", in.DeviceID)
case errors.Is(err, ErrDuplicate):
    log.Printf("mqtt: duplicate reading device=%s ignored (firmware retry)", in.DeviceID)
default:
    log.Printf("mqtt: rejected reading from %s: %v", in.DeviceID, err)
}
```

3.4 Alert Decision Logic

The state machine of Figure 2.3 is implemented as a pure function (Listing 3.3) that is exhaustively unit-tested. A thin service layer turns its decision into database writes and WebSocket events. If two readings race to open the same alert, the unique `open_marker` key described in Section 2.4 rejects the second insert.

Listing 3.3: Alert decision with hysteresis (`alerts/thresholds.go`, condensed)

```go
func decide(t Threshold, value float64, open *Alert) decision {
    if !t.Enabled {
        if open != nil {
            return decision{action: actionResolve}
        }
        return decision{action: actionNone}
    }
    var breached decision
    switch {
    case t.Min != nil && value < *t.Min:
        breached = decision{kind: "low", bound: *t.Min}
    case t.Max != nil && value > *t.Max:
        breached = decision{kind: "high", bound: *t.Max}
    }
    if breached.kind != "" {
        breached.action = actionOpen
        if open != nil {
            breached.action = actionUpdate
        }
        return breached
    }
    if open == nil {
        return decision{action: actionNone}
    }
    margin := 0.0
    if t.Min != nil && t.Max != nil {
        margin = (*t.Max - *t.Min) * hysteresisFraction // 0.05
    }
    if (open.Kind == "low" && t.Min != nil && value < *t.Min+margin) ||
        (open.Kind == "high" && t.Max != nil && value > *t.Max-margin) {
        return decision{action: actionUpdate, kind: open.Kind, bound: open.ThresholdValue}
    }
    return decision{action: actionResolve}
}
```

3.5 Time-Bucketed Aggregation

The downsampling of Equation 2.2 runs inside MySQL (Listing 3.4), so only the averaged points cross the network. The service first counts the readings in the range, using the index on `(device_id, recorded_at)`. If the count fits within N, it returns raw readings; otherwise it runs the aggregate query with the chosen bucket size.

Listing 3.4: Server-side aggregation of a history range into fixed buckets

```sql
SELECT FROM_UNIXTIME(FLOOR(UNIX_TIMESTAMP(recorded_at) / ?) * ?) AS recorded_at,
       AVG(ph) AS ph, AVG(ec_ms_cm) AS ec_ms_cm, AVG(water_temp_c) AS water_temp_c,
       AVG(air_temp_c) AS air_temp_c, AVG(humidity_pct) AS humidity_pct,
       AVG(pressure_hpa) AS pressure_hpa
FROM readings
WHERE device_id = ? AND recorded_at >= ? AND recorded_at <= ?
GROUP BY 1
ORDER BY 1;
```

3.6 Real-Time Delivery

The WebSocket hub keeps connections grouped by user. Publishing an event never touches the network directly. Each client has a buffered queue of 64 messages, drained by its own writer goroutine with a 10 s write deadline, and a client whose queue is full is disconnected instead of being allowed to stall ingestion. The server pings every 54 s and drops clients that do not answer, which removes half-open connections left behind by a mobile browser that lost signal. Browsers cannot set headers on a WebSocket request, so the short-lived access token is passed as a query parameter and verified once, at connect time.

3.7 Frontend Dashboard

The dashboard is a single-page React application with six routes: sign in, sign up, device list, device dashboard, device settings and account. One authenticated WebSocket per tab feeds every page. New readings update the cards, chart and table in place, and after a reconnect each page refetches its data to fill in anything it missed. Pages load data through a small hook that aborts superseded requests, so switching devices can never show another device's data.

The access token is held only in memory. The refresh token is kept in `localStorage`, because the API and the dashboard are on different sites, where third-party cookies are unreliable. Since refresh tokens are single-use, two browser tabs must never refresh with the same token. Refreshes are therefore serialised across tabs with the Web Locks API, re-reading the current token inside the lock (Listing 3.5).

Listing 3.5: Cross-tab-safe session refresh (`lib/session.ts`, condensed)

```ts
export function refreshSession(): Promise<Session | null> {
  if (!inflight) {
    // one refresh at a time, across all open tabs
    inflight = navigator.locks.request('pulseponics-refresh', async () => {
      const token = readRefreshToken() // re-read: another tab may have rotated it
      if (!token) { clearSession(); return null }
      const res = await fetch(apiUrl('/api/auth/refresh'), {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ refresh_token: token }),
      })
      if (res.status === 401) { clearSession(); return null }
      const session = (await res.json()) as Session
      setSession(session) // store the rotated refresh token
      return session
    }).finally(() => { inflight = null })
  }
  return inflight
}
```

3.8 Containerisation and Deployment

The stack is defined in a single Docker Compose file with five services: MySQL and the backend are bound to localhost only, Mosquitto is exposed on port 1883 for the devices, Caddy (port 80/443) is enabled by the `prod` profile, and the simulator by the `sim` profile. The backend starts only after the database and broker pass their health checks, and Mosquitto's password file and ACL are generated from environment variables when its container starts. One command, `docker compose up`, starts the stack locally. On EC2, Caddy obtains and renews a TLS certificate for an `sslip.io` hostname derived from the instance's Elastic IP, so no domain purchase is needed [11]. The backend image is built in two stages and runs as a non-root user. Database migrations are embedded in the binary and applied at startup. MySQL is tuned to fit a 1 GB t3.micro instance. The frontend is deployed separately on Vercel.

3.9 Testing and Quality Assurance

Testing was carried out at three levels:

- **Unit tests** cover the pure logic: Go tests with the race detector for the backend, Vitest for the dashboard's helper functions, and the firmware's conversion maths on the host machine.
- **An end-to-end suite** (`scripts/smoke-test.sh`) drives the live containerised stack through 38 checks: real MQTT publishes with the exact firmware payload, duplicates, unregistered devices, wrong broker passwords, alerts, threshold changes, isolation between users, WebSocket delivery, CORS and session rotation.

- **Static analysis** with `go vet`, `gofmt`, ESLint (including React's hook-purity rules) and strict TypeScript.

Chapter 4

Result

4.1 Summary of Deliverables

Table 4.1: Summary of key results achieved

Metric	Outcome
REST and WebSocket routes implemented	24
Backend code (Go, excluding tests)	approx. 3,400 lines
Frontend code (TypeScript, TSX, CSS)	approx. 3,000 lines, 6 pages
Containerised services	5 (MySQL, Mosquitto, backend, Caddy, simulator)
Automated checks passing	106 (48 backend, 8 frontend, 12 firmware, 38 end-to-end)
Median HTTP ingestion latency	2.6 ms
Median MQTT-to-dashboard latency	1.9 ms
Duplicate readings stored after a PUBACK retry	0 (3 publishes → 1 row)
Recovery after a broker restart	automatic, about 1 s

All measurements in this chapter were taken on the development machine (Apple Silicon laptop, Docker Desktop), with the client and the stack on the same host. Network latency to a cloud server is therefore not included.

4.2 Telemetry Visualisation and Alert Behaviour

To evaluate the pipeline over a realistic period, a six-hour series of 2,160 readings at the firmware's 10 s interval was ingested through the HTTP ingestion path. The pH followed a noisy random walk around 6.0, water temperature followed a slow daily swing, and a pH dosing fault was injected between 2.5 h and 3.5 h, pulling the pH towards 6.9. Figure 4.1 plots the history exactly as the dashboard receives it. The backend downsampled the 2,160 readings into 361 one-minute averages using Equation 2.2.

The alerting engine opened exactly **one** pH alert, at $t = 2.58$ h, when a reading of 6.54 first exceeded the 6.5 limit. It resolved it at $t = 3.56$ h, after the value had fallen below the hysteresis level of 6.45. Although the noisy signal crossed the 6.5 boundary several times while rising and falling, no duplicate or flapping alerts were produced.

4.3 API and End-to-End Performance

Each endpoint was called 300 times in sequence against the running stack. Figure 4.2 reports the median (p50) and 95th-percentile (p95) response times. Simple lookups complete in 1–2 ms. A raw one-hour history query, returning 360 readings, takes 3.6 ms, and aggregating six hours of data inside MySQL takes 6.3 ms. End-to-end latency was measured from the moment a reading was published to the broker until it arrived as a `reading` event on an authenticated WebSocket. Over 200 messages it was 1.9 ms at the median and never more than 4.1 ms. Login is deliberately slower (112 ms median), because bcrypt is designed to be expensive to resist password guessing; for that reason it is not shown on the chart's scale.

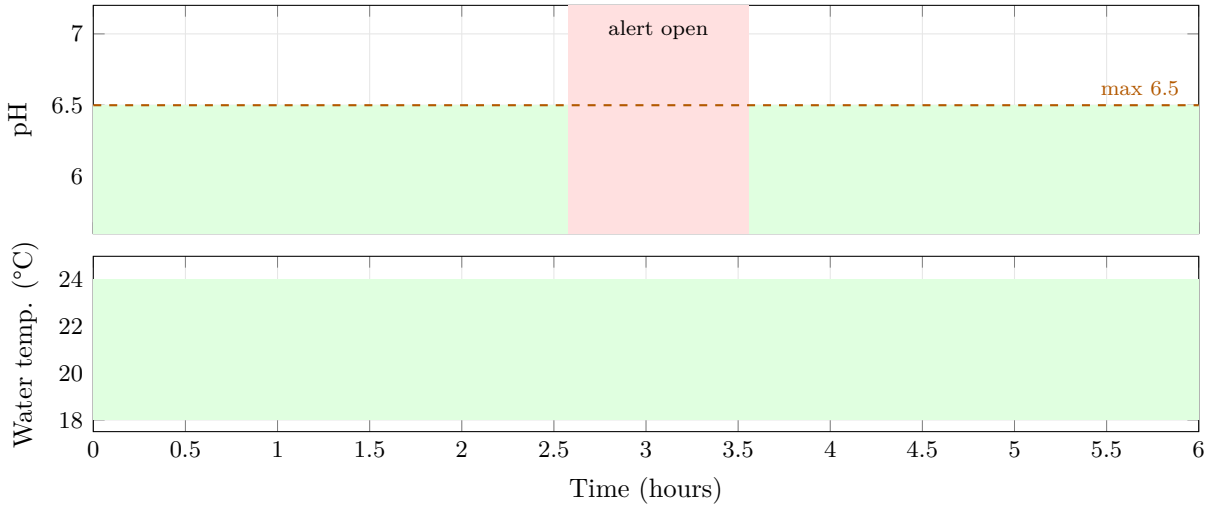


Figure 4.1: Six hours of telemetry as served by the history API (green: threshold band; red: interval during which the pH alert was open)

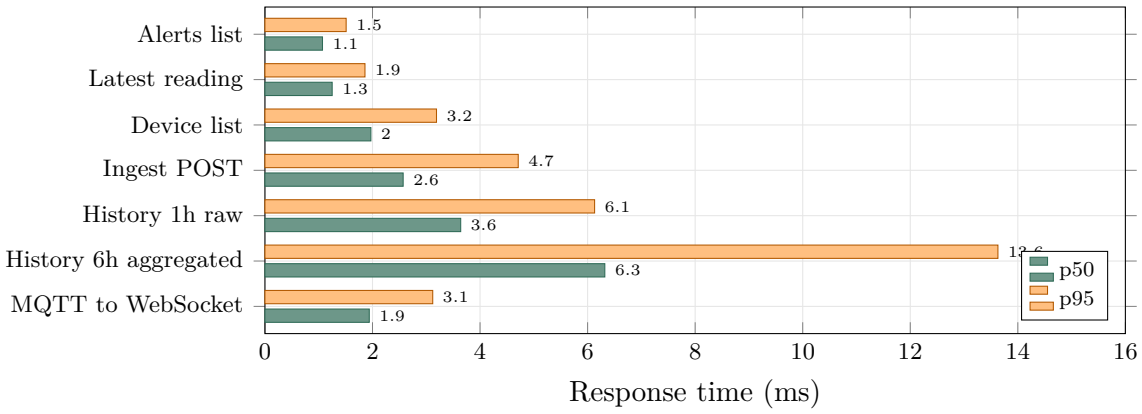


Figure 4.2: Measured response times of key operations (300 requests each; 200 messages end to end)

4.4 Reliability and Data Integrity

Several fault scenarios were reproduced against the running stack:

- **Duplicate delivery:** the same reading published three times, as a firmware PUBACK retry would, was stored once. The other two were logged as duplicates and were neither broadcast nor alerted on.
- **Broker restart:** the backend logged the lost connection, reconnected and resubscribed within about one second, and the next reading was stored.
- **Unregistered or spoofed devices:** readings from unregistered IDs, and payloads whose `device_id` differed from the topic, were dropped. A wrong broker password was refused by Mosquitto.
- **User isolation:** a second account received `404` for the first account's device, and received none of its WebSocket events.

4.5 Dashboard

Figure 4.3 shows the device dashboard during a live alert. The pH card is flagged *Too high*, the chart shows the threshold band, the alert can be acknowledged, and recent out-of-range values are highlighted. Figure 4.4 shows the per-device threshold editor, which gives immediate validation feedback and applies changes to the latest reading at once, and the responsive layout on a phone.

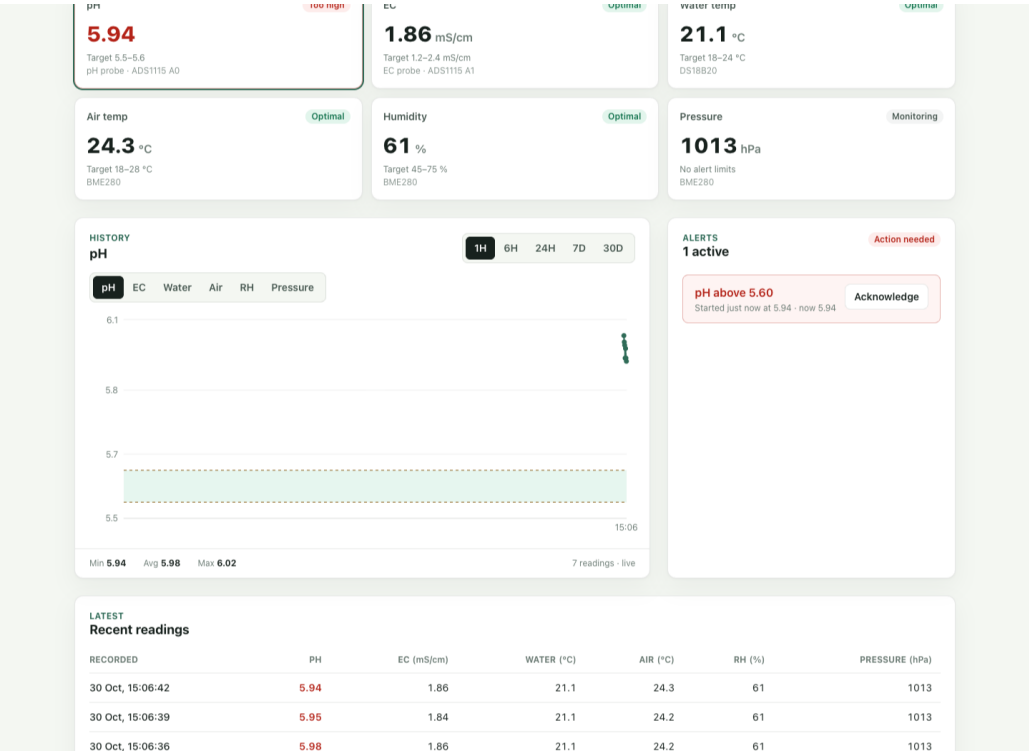
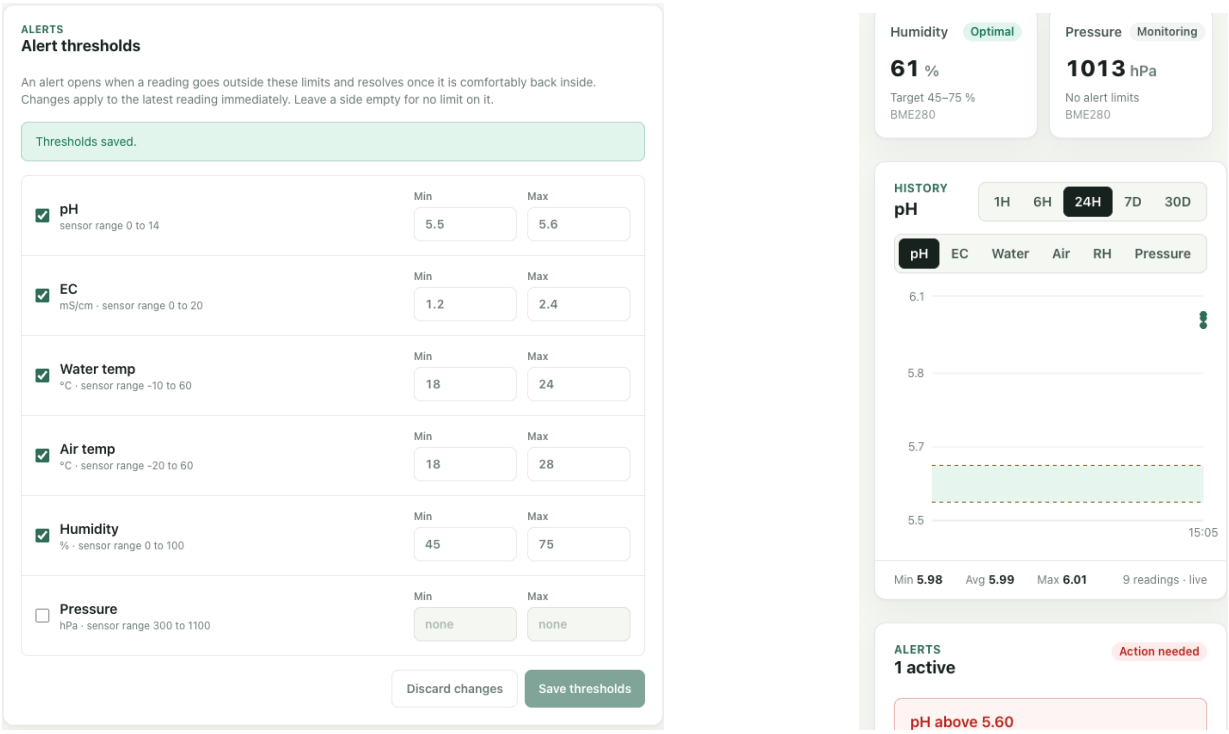


Figure 4.3: Device dashboard with live readings, history chart and an open pH alert

4.6 Test Results



(a) Per-device alert threshold editor **(b)** Mobile layout

Figure 4.4: Threshold configuration and responsive design of the dashboard

Table 4.2: Automated test results

Suite	Scope	Cases	Result
Backend unit	validation, ingestion flow, bucket sizing, alert decisions and lifecycle, tokens, refresh rotation and reuse, CORS, migrations	48	pass
Frontend unit	metric state, formatting, chart gap detection, axis scaling	8	pass
Firmware unit	pH/EC conversion, temperature compensation, range checks	12	pass
End-to-end	full stack over real MQTT, HTTP and WebSocket (Section 3.9)	38	pass
Static analysis	`go vet`, `gofmt`, ESLint, strict TypeScript	–	clean

Chapter 5

Discussion

5.1 Engineering Reliable Ingestion

The central challenge of the software platform was turning MQTT's at-least-once delivery and an intermittently connected device into a clean, gap-aware time series. Three issues surfaced during development, and each was fixed at its root rather than patched over:

- **Duplicates.** The firmware's retry produced duplicate rows. Instead of adding deduplication logic to the service, the natural key `(device_id, recorded_at)` was made unique, which makes the insert itself idempotent.
- **Silent loss of subscriptions.** After a broker restart, ingestion stopped while the service still appeared healthy. Moving the subscription into the on-connect handler fixed it, and the scenario is now part of the verification.
- **A reserved-word bug.** Running the database migration in a container revealed that `last_value` is a reserved word in MySQL 8. It would have failed on the first deployment had it not been tested against the real database engine. Migrations were also made re-runnable for this reason.

5.2 Design Trade-offs Considered

- **MySQL versus a time-series database.** A dedicated time-series database offers built-in downsampling, but MySQL was fixed by the proposal and is sufficient at this scale: a composite index plus in-database bucketing keeps six-hour queries under 7 ms.
- **Shared versus per-device broker credentials.** All devices share one MQTT login, while ownership is enforced by the backend's registry and its topic-to-payload check. Per-device credentials would need the broker's dynamic-security plugin, which was not worth the complexity for a single-tank deployment.
- **Plain MQTT on port 1883 versus TLS on 8883.** TLS on the ESP32 needs an embedded root certificate and costs memory. Password authentication over plain TCP was accepted for the demonstration phase and is listed as future work.
- **Refresh token in `localStorage` versus an httpOnly cookie.** A cookie is better protected against cross-site scripting, but it is unreliable across the Vercel and EC2 sites. The risk is reduced by short-lived access tokens kept in memory, rotation, and reuse detection.

5.3 Limitations

The analog pH and EC probes and the BME280 are not yet integrated, so the data analysed in this report comes from a simulator that uses the firmware's exact contract, not from a physical tank. Performance was measured on a single machine without network latency or concurrent load. Alerts are currently delivered in-app only, and device offline detection from the MQTT status topic is not yet implemented.

Chapter 6

Conclusion

The first half of the capstone project has delivered a complete, tested software platform for real-time hydroponics monitoring:

- an authenticated MQTT broker;

- a Go backend with secure multi-user accounts, a device registry, idempotent ingestion, server-side downsampling, a hysteresis-based alerting engine and a real-time WebSocket feed;
- a responsive React dashboard;
- a containerised deployment with automatic HTTPS.

The platform accepts the firmware's existing payload without modification, so the remaining hardware work plugs directly into it. Measured latencies are in the low milliseconds, the pipeline remained correct under duplicate delivery, broker restarts and spoofed devices, and all 106 automated checks pass.

6.1 Future Scope

The plan for the second half of the semester is:

1. Implement the ADS1115 and BME280 drivers, and perform two-point calibration of the pH (4.00/7.00) and EC (1.413/12.88 mS/cm) probes with temperature compensation.
2. Deploy the stack on AWS EC2 with the dashboard on Vercel, and run a multi-day field trial on a real tank.
3. Add external notifications (Telegram or email) and debounced offline alerts based on the MQTT status topic.
4. Enable MQTT over TLS (port 8883), and evaluate behaviour under concurrent load with multiple devices.

References

[1] H. M. Resh, *Hydroponic Food Production*, 7th ed. Boca Raton, FL: CRC Press, 2012.

[2] OASIS, *MQTT Version 3.1.1*, OASIS Standard, 2014. `https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html`

[3] Eclipse Foundation, *Eclipse Mosquitto Documentation*. `https://mosquitto.org/documentation/`

[4] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[5] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, 2011.

[6] OWASP Foundation, *Authentication and Session Management Cheat Sheets*. `https://cheatsheetseries.owasp.org/`

[7] The Go Authors, *The Go Programming Language Documentation*. `https://go.dev/doc/`

[8] Oracle, *MySQL 8.4 Reference Manual*. `https://dev.mysql.com/doc/refman/8.4/en/`

[9] Meta Open Source, *React Documentation*. `https://react.dev/`

[10] Docker Inc., *Docker Compose Documentation*. `https://docs.docker.com/compose/`

[11] Caddy Project, *Automatic HTTPS*. `https://caddyserver.com/docs/automatic-https`

[12] DFRobot, *Gravity: Analog pH Sensor/Meter Kit V2 and Analog EC Meter V2 – Product Wiki*. `https://wiki.dfrobot.com/`